

CineAustral — Documentación del Sistema y Diseño de Software

Plataforma de gestión cinematográfica y reservas

Ítem	Detalle
Frontend	React 19, Vite, Tailwind CSS, Axios, Lucide React
Backend	Java 21, Spring Boot 4.0.6, Spring Data JPA, Hibernate, Spring Security, Lombok
Base de datos	Oracle Database (19c, 21c y 23c Free), driver oficial ojdbc11
Integración externa	The Movie Database (TMDB) API v3
Complejo	2 salas físicas: <i>Sala Lobo Marino</i> y <i>Sala Pingüino Magallanes</i> (100 butacas cada una)

Introducción

CineAustral es un sistema web para la administración de un complejo cinematográfico de dos salas y la reserva de butacas en línea. Permite a los administradores gestionar la cartelera, las funciones, las salas y las butacas, y a los clientes consultar la cartelera, elegir asientos en un mapa interactivo y administrar sus reservas.

Este documento describe el sistema desde cuatro ejes: **(1)** requerimientos funcionales, **(2)** arquitectura cliente-servidor, **(3)** modelo de base de datos y persistencia, y **(4)** patrones de diseño aplicados.

Actores del sistema

Actor	Descripción
Visitante	Persona sin sesión iniciada. Puede ver la cartelera y registrarse.
Cliente	Usuario autenticado con rol CLIENTE. Reserva y gestiona sus propias reservas.
Administrador	Usuario autenticado con rol ADMIN. Gestiona el contenido y la operación del cine.

1. Requerimientos Funcionales (RF)

1.1 Convenciones

Los identificadores siguen el formato RF - <MÓDULO> - <NN>:

Prefijo	Módulo
AUT	Autenticación y control de acceso
USR	Gestión de usuarios
PEL	Películas e integración TMDB
FUN	Funciones
SAL	Salas y butacas
RES	Reservas
CAS	Cancelación en cascada por fuerza mayor
DASH	Dashboard administrativo

1.2 Módulo de Autenticación y Control de Acceso (AUT)

RF-AUT-01 — Registro de usuario

- **Actor principal:** Visitante
- **Descripción:** Permite a una persona crear una cuenta en el sistema para poder realizar reservas.
- **Reglas de negocio y validaciones:**
 - Se requieren nombre, apellido, correo electrónico y contraseña.
 - El correo electrónico es único en el sistema (UNIQUE); no se admiten cuentas duplicadas.
 - Las cuentas creadas por autoregistro reciben el rol CLIENTE.
- **Resultado / Salida:** Cuenta creada y confirmación al usuario; en caso de correo ya registrado o datos inválidos, mensaje de error con el código HTTP correspondiente.

RF-AUT-02 — Inicio de sesión

- **Actor principal:** Visitante
- **Descripción:** Autentica al usuario mediante correo y contraseña a través de Spring Security.
- **Reglas de negocio y validaciones:**
 - Las credenciales se validan contra los datos almacenados; ante credenciales inválidas no se revela cuál de los dos datos es incorrecto.
 - La contraseña nunca se incluye en las respuestas de la API.
- **Resultado / Salida:** Sesión autenticada y datos básicos del usuario (identificador, nombre, rol) para que el cliente configure su estado de autenticación.

RF-AUT-03 — Control de acceso por rol

- **Actor principal:** Cliente / Administrador
- **Descripción:** Restringe el acceso a operaciones y pantallas según el rol (ADMIN, CLIENTE).
- **Reglas de negocio y validaciones:**
 - Las operaciones de administración (películas, funciones, salas, butacas, usuarios, dashboard) solo son accesibles para ADMIN.
 - Las operaciones de reserva requieren una sesión autenticada.
 - Un cliente solo puede consultar, modificar y cancelar **sus propias** reservas.
 - La SPA oculta las rutas no autorizadas, y el servidor aplica la restricción de forma definitiva.
- **Resultado / Salida:** Acceso concedido o rechazo (401 / 403).

RF-AUT-04 — Gestión del estado de autenticación en el cliente

- **Actor principal:** Cliente / Administrador
- **Descripción:** La SPA mantiene el estado de sesión del usuario y lo utiliza para habilitar o deshabilitar rutas y acciones.
- **Reglas de negocio y validaciones:**
 - Al cerrar sesión, el estado de autenticación se elimina del cliente.
 - La interfaz muestra opciones distintas según el rol.
- **Resultado / Salida:** Navegación coherente con el rol del usuario.

1.3 Módulo de Gestión de Usuarios (USR)

RF-USR-01 — Administración de usuarios

- **Actor principal:** Administrador
- **Descripción:** Permite listar y consultar usuarios del sistema, incluyendo la asignación de rol.
- **Reglas de negocio y validaciones:**
 - El correo electrónico debe mantenerse único.
 - Los roles válidos son ADMIN y CLIENTE.
 - Los datos sensibles (contraseña) no se exponen en las respuestas.
- **Resultado / Salida:** Listado y fichas de usuarios actualizados.

1.4 Módulo de Películas e Integración TMDB (PEL)

RF-PEL-01 — Búsqueda de películas en TMDB

- **Actor principal:** Administrador
- **Descripción:** Permite buscar en vivo películas en la API oficial v3 de The Movie Database para evitar la carga manual de datos.
- **Reglas de negocio y validaciones:**
 - La consulta se realiza desde el servidor, que encapsula la comunicación con el servicio externo.
 - El administrador puede seleccionar un resultado para precargar el formulario.
- **Resultado / Salida:** Lista de coincidencias con título, sinopsis, duración, puntuación y póster.

RF-PEL-02 — Importación de metadatos cinematográficos

- **Actor principal:** Administrador
- **Descripción:** Autocompleta el alta de una película con los metadatos obtenidos de TMDB.
- **Reglas de negocio y validaciones:**
 - Se importan título, sinopsis, duración (minutos), puntuación y póster en alta resolución (imagen_url).
 - La duración es obligatoria (NOT NULL) porque determina el horario de finalización de las funciones.
 - Los datos importados pueden ser revisados y editados antes de guardar.
- **Resultado / Salida:** Película registrada en el catálogo local.

RF-PEL-03 — Alta, edición y baja de películas

- **Actor principal:** Administrador
- **Descripción:** Gestión completa (CRUD) del catálogo de películas, incluyendo la carga manual cuando no se utilice TMDB.
- **Reglas de negocio y validaciones:**
 - Campos obligatorios: título y duración en minutos.
 - Campos opcionales: género, sinopsis, puntuación, clasificación, URL de imagen.
 - Una película con funciones asociadas no puede eliminarse sin comprometer la integridad referencial; en ese caso se recomienda retirarla de cartelera.
- **Resultado / Salida:** Catálogo actualizado.

RF-PEL-04 — Control de estado en cartelera

- **Actor principal:** Administrador
- **Descripción:** Permite activar o retirar una película de la cartelera mediante el indicador enCartelera.
- **Reglas de negocio y validaciones:**
 - Por defecto, una película nueva queda en cartelera (`en_cartelera = 1`).
 - Solo las películas con `enCartelera == true` pueden programarse en nuevas funciones.
- **Resultado / Salida:** La película aparece o desaparece de la cartelera pública y del selector de programación de funciones.

RF-PEL-05 — Consulta de cartelera

- **Actor principal:** Visitante / Cliente
- **Descripción:** Muestra las películas en cartelera con póster, sinopsis, género, duración, puntuación, clasificación y funciones disponibles.
- **Reglas de negocio y validaciones:**
 - Solo se listan películas activas en cartelera.
 - Solo se muestran funciones futuras en estado activo.
- **Resultado / Salida:** Cartelera en formato de tarjetas con acceso a la reserva.

1.5 Módulo de Funciones (FUN)**RF-FUN-01 — Programación de funciones**

- **Actor principal:** Administrador
- **Descripción:** Permite programar una función asignando película, sala, fecha y hora de inicio y precio por asiento.
- **Reglas de negocio y validaciones:**
 - La película debe estar activa en cartelera (`enCartelera == true`).
 - La sala seleccionada debe encontrarse disponible.
 - La duración de la función se toma de la película (`duracion_minutos`).
 - El precio por asiento es obligatorio y positivo.
 - La función se crea con estado ACTIVA.
- **Resultado / Salida:** Función registrada y visible para reserva.

RF-FUN-02 — Validación de franja horaria permitida

- **Actor principal:** Administrador
- **Descripción:** Garantiza que todas las funciones se programen dentro del horario operativo del cine.
- **Reglas de negocio y validaciones:**
 - El **inicio** debe ser mayor o igual a las **18:00 hs.**
 - El **fin** (inicio + duración) debe ser menor o igual a las **00:00 hs.**
 - Se rechaza toda función que comience antes o termine después de ese rango.
- **Resultado / Salida:** Función aceptada o rechazada con mensaje explicativo.

RF-FUN-03 — Validación de anti-solapamiento

- **Actor principal:** Administrador
- **Descripción:** Evita que dos funciones ocupen simultáneamente la misma sala física.
- **Reglas de negocio y validaciones:**
 - En una misma sala, el intervalo [inicio, fin] de la nueva función no puede superponerse con el de ninguna otra función activa del mismo día.
 - Las funciones canceladas no participan de la validación.
 - La verificación y el guardado se ejecutan en una única transacción.
- **Resultado / Salida:** Función aceptada o rechazada por conflicto de horario.

RF-FUN-04 — Consulta de funciones y estado calculado

- **Actor principal:** Administrador / Cliente / Visitante
- **Descripción:** Lista las funciones por película, sala o fecha, indicando su estado.
- **Reglas de negocio y validaciones:**
 - El estado persistido es ACTIVA o CANCELADA.
 - El estado FINALIZADA se **calcula** dinámicamente cuando el fin de la función es anterior al momento actual; no se almacena.
 - Los clientes solo visualizan funciones futuras y activas.
- **Resultado / Salida:** Listado de funciones con su estado vigente.

RF-FUN-05 — Cancelación de función

- **Actor principal:** Administrador
- **Descripción:** Permite cancelar una función por razones operativas.
- **Reglas de negocio y validaciones:**
 - La función pasa a estado CANCELADA.
 - Todas las reservas activas asociadas pasan automáticamente a CANCELADA (ver RF-CAS-01).
- **Resultado / Salida:** Función cancelada y reservas afectadas actualizadas.

1.6 Módulo de Salas y Butacas (SAL)**RF-SAL-01 — Gestión de salas físicas**

- **Actor principal:** Administrador
- **Descripción:** Consulta y administra las salas del complejo (*Sala Lobo Marino* y *Sala Pingüino Magallanes*).
- **Reglas de negocio y validaciones:**
 - Estados válidos: DISPONIBLE y NO_DISPONIBLE.
 - Solo las salas DISPONIBLE admiten programar nuevas funciones.
- **Resultado / Salida:** Estado actualizado de cada sala.

RF-SAL-02 — Habilitación e inhabilitación de salas por mantenimiento

- **Actor principal:** Administrador

- **Descripción:** Permite pasar una sala a NO_DISPONIBLE, indicando una fecha límite o rango de mantenimiento.
- **Reglas de negocio y validaciones:**
 - Al inhabilitar una sala con fecha límite, se cancelan en cascada todas las funciones de esa sala comprendidas en el rango afectado (ver RF-CAS-02).
 - Una sala inhabilitada no puede recibir nuevas funciones.
 - Al rehabilitarla, vuelve a estado DISPONIBLE.
- **Resultado / Salida:** Sala actualizada y resumen de funciones y reservas canceladas.

RF-SAL-03 — Puesta de butacas en mantenimiento

- **Actor principal:** Administrador
- **Descripción:** Permite marcar butacas individuales como MANTENIMIENTO o devolverlas a DISPONIBLE.
- **Reglas de negocio y validaciones:**
 - Las butacas en mantenimiento no pueden seleccionarse en una nueva reserva.
 - Cada butaca se identifica por fila y número dentro de su sala.
- **Resultado / Salida:** Mapa de sala actualizado.

RF-SAL-04 — Distribución de la sala en bloques

- **Actor principal:** Administrador / Cliente
- **Descripción:** Presenta la sala organizada en tres bloques de auditorio sobre un mapa interactivo.
- **Reglas de negocio y validaciones:**
 - Cada sala dispone de **100 butacas**:

Bloque	Filas	Asientos	Butacas
Izquierdo	B a F	1 a 4	20
Central	A a F	5 a 14	60
Derecho	B a F	15 a 18	20

- Cada butaca se muestra como disponible, ocupada, en mantenimiento o seleccionada.
- **Resultado / Salida:** Mapa visual con el estado de cada butaca para la función consultada.

1.7 Módulo de Reservas (RES)

RF-RES-01 — Consulta de disponibilidad de butacas

- **Actor principal:** Cliente
- **Descripción:** Para una función determinada, informa qué butacas están libres, ocupadas o en mantenimiento.
- **Reglas de negocio y validaciones:**
 - Una butaca está ocupada si pertenece a una reserva en estado ACTIVA de esa función.
 - Las butacas en MANTENIMIENTO se muestran como no seleccionables.
- **Resultado / Salida:** Mapa de butacas con el estado actualizado.

RF-RES-02 — Creación de reserva

- **Actor principal:** Cliente

- **Descripción:** Permite reservar una o más butacas para una función.
- **Reglas de negocio y validaciones:**
 - **Vigencia:** solo se reservan funciones futuras, con un máximo de **7 días** de anticipación.
 - **No colisión:** ningún asiento solicitado puede estar ocupado por otra reserva ACTIVA de la misma función; la verificación y el guardado se realizan en una única transacción (@Transactional) para evitar que dos personas reserven la misma butaca.
 - Las butacas deben pertenecer a la sala de la función y no estar en mantenimiento.
 - La función debe estar ACTIVA.
 - **Total dinámico:** $\text{precio_total} = \text{precio_por_asiento} \times \text{cantidad de butacas}$.
 - La reserva se crea con estado ACTIVA y registra fecha de reserva y cliente.
- **Resultado / Salida:** Reserva confirmada con el detalle de butacas y el total.

RF-RES-03 — Modificación de reserva

- **Actor principal:** Cliente
- **Descripción:** Permite al cliente modificar su reserva (por ejemplo, la selección de butacas).
- **Reglas de negocio y validaciones:**
 - Solo es posible si faltan **más de 10 minutos** para el inicio de la función.
 - No se puede modificar una reserva ya MODIFICADA ni una CANCELADA.
 - Se aplican nuevamente las validaciones de no colisión y de butacas disponibles, y se recalcula el total.
 - Tras la modificación, la reserva pasa a estado MODIFICADA.
- **Resultado / Salida:** Reserva actualizada o rechazo con el motivo.

RF-RES-04 — Cancelación de reserva por el cliente

- **Actor principal:** Cliente
- **Descripción:** Permite cancelar una reserva propia.
- **Reglas de negocio y validaciones:**
 - Solo es posible si faltan **más de 10 minutos** para el inicio de la función.
 - No se puede cancelar una reserva ya CANCELADA.
 - Al cancelarse, las butacas quedan nuevamente disponibles.
- **Resultado / Salida:** Reserva en estado CANCELADA.

RF-RES-05 — Historial de reservas del cliente

- **Actor principal:** Cliente
- **Descripción:** Lista las reservas del cliente con película, sala, fecha y hora, butacas, total y estado.
- **Reglas de negocio y validaciones:**
 - Estados posibles: ACTIVA, CANCELADA, COMPLETADA, MODIFICADA.
 - Cada cliente solo ve sus propias reservas.
- **Resultado / Salida:** Listado de reservas con las acciones permitidas según el estado y la antelación.

RF-RES-06 — Consulta administrativa de reservas

- **Actor principal:** Administrador

- **Descripción:** Permite al administrador consultar las reservas del sistema por función, cliente o estado.
- **Reglas de negocio y validaciones:** Acceso restringido al rol ADMIN.
- **Resultado / Salida:** Listado de reservas con filtros.

1.8 Cancelación en Cascada por Fuerza Mayor (CAS)

RF-CAS-01 — Cancelación en cascada por cancelación de función

- **Actor principal:** Administrador
- **Descripción:** Al cancelar una función, el sistema cancela automáticamente todas sus reservas activas.
- **Reglas de negocio y validaciones:**
 - Toda reserva ACTIVA de la función pasa a CANCELADA.
 - La operación es atómica: o se aplican todos los cambios, o ninguno.
 - Esta cancelación no está sujeta a la restricción de los 10 minutos, ya que se origina en el cine.
- **Resultado / Salida:** Función y reservas en estado CANCELADA.

RF-CAS-02 — Cancelación en cascada por mantenimiento de sala

- **Actor principal:** Administrador
- **Descripción:** Al pasar una sala a NO_DISPONIBLE con una fecha límite o rango de mantenimiento, se cancelan las funciones afectadas y sus reservas.
- **Reglas de negocio y validaciones:**
 - Se cancelan todas las funciones de la sala dentro del rango de mantenimiento.
 - Todas las reservas activas de esas funciones pasan a CANCELADA.
 - Todo el proceso se ejecuta en una única transacción.
- **Resultado / Salida:** Sala en mantenimiento, funciones canceladas y reservas canceladas.

1.9 Módulo de Dashboard Administrativo (DASH)

RF-DASH-01 — Recaudación acumulada

- **Actor principal:** Administrador
- **Descripción:** Muestra el total recaudado por las reservas.
- **Reglas de negocio y validaciones:** No se computan las reservas CANCELADAS.
- **Resultado / Salida:** Monto total acumulado.

RF-DASH-02 — Entradas vendidas

- **Actor principal:** Administrador
- **Descripción:** Informa la cantidad total de entradas (butacas) vendidas.
- **Reglas de negocio y validaciones:** Se cuentan únicamente las butacas de reservas no canceladas.
- **Resultado / Salida:** Cantidad de entradas vendidas.

RF-DASH-03 — Ocupación promedio por sala

- **Actor principal:** Administrador
- **Descripción:** Calcula el porcentaje promedio de ocupación de cada sala.

- **Reglas de negocio y validaciones:** Se calcula como butacas reservadas sobre la capacidad de la sala (100), promediado entre sus funciones.
- **Resultado / Salida:** Porcentaje de ocupación por sala (*Lobo Marino* y *Pingüino Magallanes*).

RF-DASH-04 — Gráfico de ventas de los últimos 7 días

- **Actor principal:** Administrador
- **Descripción:** Presenta la evolución diaria de las ventas durante la última semana.
- **Reglas de negocio y validaciones:** Una serie diaria de 7 puntos; los días sin ventas se muestran en cero.
- **Resultado / Salida:** Gráfico de ventas diarias.

RF-DASH-05 — Registro de actividad reciente

- **Actor principal:** Administrador
- **Descripción:** Lista los eventos más recientes del sistema (reservas creadas, modificadas o canceladas, entre otros).
- **Reglas de negocio y validaciones:** Orden cronológico descendente.
- **Resultado / Salida:** Feed de actividad reciente.

2. Arquitectura del Sistema (Modelo Cliente-Servidor)

2.1 Visión general

CineAustral adopta un modelo **cliente-servidor desacoplado**. El cliente (SPA en React) y el servidor (API REST en Spring Boot) se despliegan y evolucionan de forma independiente, y su único punto de contacto es el contrato HTTP/JSON de la API. Esto permite reemplazar o ampliar el cliente (por ejemplo, una aplicación móvil) sin modificar la lógica de negocio, y mantener las reglas del dominio centralizadas y protegidas en el servidor.

2.2 Capa de Presentación (Cliente)

Single Page Application construida con **React 19** y **Vite**, y estilizada con **Tailwind CSS**.

Responsabilidades:

- **Experiencia de usuario:** cartelera en tarjetas, formularios de administración y panel con gráficos.
- **Mapa interactivo de butacas:** renderizado reactivo de los tres bloques de la sala (izquierdo, central y derecho), con estados visuales por butaca (disponible, ocupada, mantenimiento, seleccionada) que se actualizan sin recargar la página.
- **Gestión del estado de autenticación:** mantiene la sesión del usuario y habilita rutas y acciones según el rol.
- **Consumo de la API REST:** peticiones asíncronas mediante **Axios**, con manejo centralizado de errores y de los códigos de estado HTTP.

2.3 Capa de Lógica de Negocio y Servicios (Servidor)

API RESTful desarrollada con **Spring Boot 4.0.6** sobre **Java 21**, asegurada con **Spring Security**.

Internamente se organiza en tres capas:

Capa	Responsabilidad
Controladores (Controllers)	Exponen los endpoints REST, enrutan las peticiones, validan la entrada y traducen los resultados a códigos de estado HTTP. No contienen reglas de negocio.
Servicios (Services)	Encapsulan las reglas de negocio (horarios, anti-solapamiento, colisión de asientos, políticas de cancelación y modificación, cascadas), garantizan la atomicidad con <code>@Transactional</code> e integran clientes externos como la API de TMDB.
Repositorios (Repositories)	Abstraen el acceso a datos con Spring Data JPA: operaciones CRUD, consultas derivadas y consultas personalizadas.

2.4 Capa de Datos (Persistencia)

Motor relacional **Oracle Database** (19c, 21c y 23c Free), accedido mediante el driver oficial **ojdbc11**. Es responsable de la integridad referencial, de las claves generadas por identidad y del almacenamiento transaccional de usuarios, salas, butacas, películas, funciones y reservas.

2.5 Integración externa

El servidor consume la **API v3 de TMDB** para la búsqueda e importación de metadatos y pósters. El cliente nunca se comunica directamente con TMDB: el servidor actúa como intermediario, lo que protege las credenciales de acceso y permite normalizar los datos antes de devolverlos.

2.6 Diagrama conceptual de arquitectura cliente-servidor

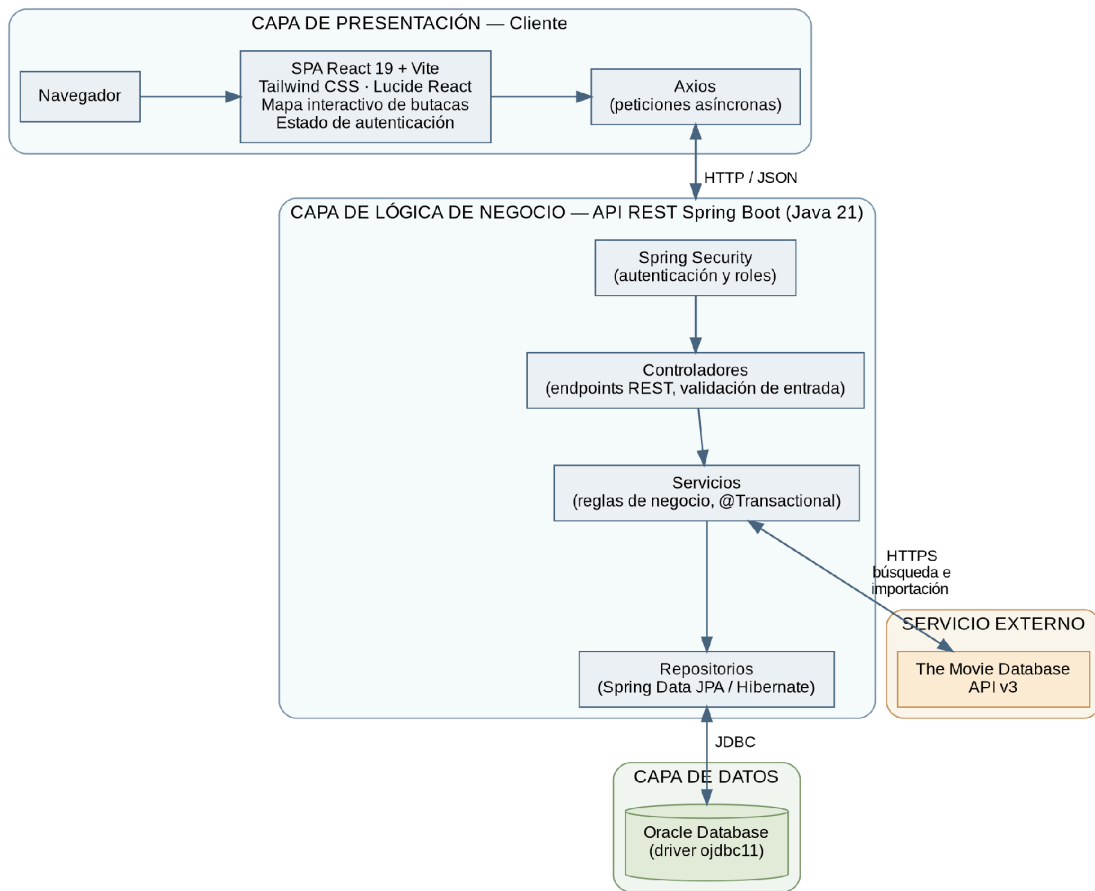


Figura 1. Diagrama conceptual de arquitectura cliente-servidor

3. Modelo de Base de Datos y Persistencia

3.1 Diagrama Entidad-Relación (MER)

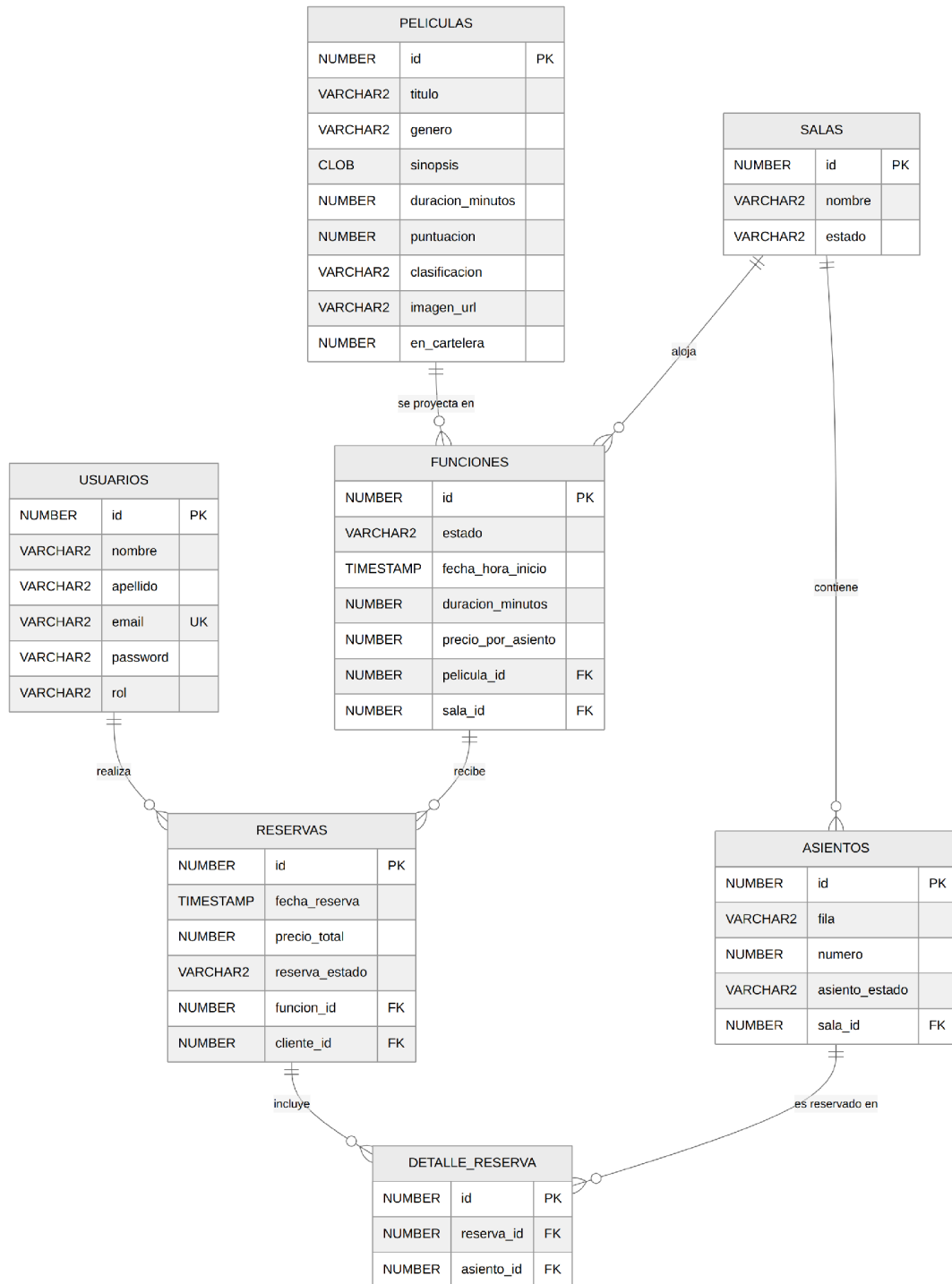


Figura 2. Diagrama Entidad-Relación (MER) de CineAustral

Cardinalidades

Relación	Cardinalidad	Descripción
usuarios — reservas	1 : N	Un usuario (cliente) realiza muchas reservas.
salas — asientos	1 : N	Una sala contiene 100 butacas.

Relación	Cardinalidad	Descripción
salas — funciones	1 : N	Una sala aloja muchas funciones a lo largo del tiempo.
películas — funciones	1 : N	Una película se proyecta en muchas funciones.
funciones — reservas	1 : N	Una función recibe muchas reservas.
reservas — detalle_reserva	1 : N	Una reserva incluye una o más butacas.
asientos — detalle_reserva	1 : N	Una butaca puede figurar en muchas reservas (en distintas funciones).

La relación **muchos a muchos** entre reservas y asientos se descompone mediante la tabla asociativa `detalle_reserva`.

3.2 Diccionario de Datos

Tabla usuarios

Almacena las cuentas del sistema.

Campo	Tipo	Restricciones	Descripción
id	NUMBER	PRIMARY KEY, GENERATED BY DEFAULT AS IDENTITY	Identificador único del usuario.
nombre	VARCHAR2(255)	NOT NULL	Nombre de pila.
apellido	VARCHAR2(255)	NOT NULL	Apellido.
email	VARCHAR2(255)	NOT NULL, UNIQUE	Correo electrónico; se usa como credencial de acceso.
password	VARCHAR2(255)	NOT NULL	Credencial de acceso del usuario.
rol	VARCHAR2(50)	NOT NULL	Rol del usuario: ADMIN o CLIENTE (enum UsuarioRol).

Tabla salas

Salas físicas del complejo.

Campo	Tipo	Restricciones	Descripción
id	NUMBER	PRIMARY KEY, GENERATED BY DEFAULT AS IDENTITY	Identificador de la sala.
nombre	VARCHAR2(255)	NOT NULL	Nombre de la sala (<i>Sala Lobo Marino, Sala Pingüino Magallanes</i>).
estado	VARCHAR2(50)	NOT NULL	DISPONIBLE o NO_DISPONIBLE (enum SalaEstado).

Tabla asientos

Butacas individuales de cada sala.

Campo	Tipo	Restricciones	Descripción
id	NUMBER	PRIMARY KEY, GENERATED BY DEFAULT AS IDENTITY	Identificador de la butaca.
fila	VARCHAR2(10)	NOT NULL	Fila de la butaca (A a F).

Campo	Tipo	Restricciones	Descripción
numero	NUMBER(5)	NOT NULL	Número de asiento dentro de la fila (1 a 18).
asiento_estado	VARCHAR2(50)	NOT NULL	DISPONIBLE o MANTENIMIENTO (enum AsientoEstado).
sala_id	NUMBER	NOT NULL, FOREIGN KEY → salas(id), ON DELETE CASCADE	Sala a la que pertenece.

Tabla películas

Catálogo de películas.

Campo	Tipo	Restricciones	Descripción
id	NUMBER	PRIMARY KEY, GENERATED BY DEFAULT AS IDENTITY	Identificador de la película.
titulo	VARCHAR2(255)	NOT NULL	Título.
genero	VARCHAR2(255)	—	Género o géneros.
sinopsis	CLOB	—	Resumen argumental de longitud variable.
duracion_minutos	NUMBER(5)	NOT NULL	Duración en minutos; define el fin de las funciones.
puntuacion	NUMBER(3,1)	—	Valoración (por ejemplo, 7.8), proveniente de TMDB.
clasificacion	VARCHAR2(50)	—	Clasificación por edades.
imagen_url	VARCHAR2(500)	—	URL del póster.
en_cartelera	NUMBER(1)	NOT NULL, DEFAULT 1	Indicador booleano (1 = en cartelera, 0 = fuera).

Tabla funciones

Proyecciones programadas.

Campo	Tipo	Restricciones	Descripción
id	NUMBER	PRIMARY KEY, GENERATED BY DEFAULT AS IDENTITY	Identificador de la función.
estado	VARCHAR2(50)	NOT NULL, DEFAULT 'ACTIVA'	ACTIVA o CANCELADA (enum FuncionEstado). FINALIZADA se calcula cuando el fin es anterior al momento actual.
fecha_hora_inicio	TIMESTAMP	NOT NULL	Fecha y hora de comienzo.
duracion_minutos	NUMBER(5)	NOT NULL	Duración de la función en minutos.
precio_por_asiento	NUMBER(10,2)	NOT NULL	Precio unitario de la entrada.
pelicula_id	NUMBER	NOT NULL, FOREIGN KEY → peliculas(id)	Película proyectada.

Campo	Tipo	Restricciones	Descripción
sala_id	NUMBER	NOT NULL, FOREIGN KEY → salas(id)	Sala donde se proyecta.

Tabla reservas

Encabezado de cada reserva.

Campo	Tipo	Restricciones	Descripción
id	NUMBER	PRIMARY KEY, GENERATED BY DEFAULT AS IDENTITY	Identificador de la reserva.
fecha_reserva	TIMESTAMP	NOT NULL	Momento en que se efectuó la reserva.
precio_total	NUMBER(10,2)	NOT NULL	Total (precio_por_asiento × butacas).
reserva_estado	VARCHAR2(50)	—	ACTIVA, CANCELADA, COMPLETADA o MODIFICADA (enum ReservaEstado).
funcion_id	NUMBER	NOT NULL, FOREIGN KEY → funciones(id)	Función reservada.
cliente_id	NUMBER	NOT NULL, FOREIGN KEY → usuarios(id)	Cliente que reserva.

Tabla detalle_reserva

Tabla asociativa entre reservas y butacas.

Campo	Tipo	Restricciones	Descripción
id	NUMBER	PRIMARY KEY, GENERATED BY DEFAULT AS IDENTITY	Identificador de la línea de detalle.
reserva_id	NUMBER	FOREIGN KEY → reservas(id), ON DELETE CASCADE	Reserva a la que pertenece.
asiento_id	NUMBER	FOREIGN KEY → asientos(id)	Butaca reservada.

Observaciones de diseño

- La unicidad de una butaca por función **no** se impone con una restricción UNIQUE en la base de datos, porque depende del estado de la reserva y de la función; se garantiza en la capa de servicios dentro de una transacción (RF-RES-02).
- El estado FINALIZADA de una función no es una columna: se deriva en tiempo de consulta comparando el fin de la función con la fecha actual.
- Las columnas reserva_id y asiento_id de detalle_reserva y reserva_estado de reservas admiten nulos en el esquema; la aplicación siempre las completa.

3.3 Enumeraciones del dominio

Enum	Valores
UsuarioRol	ADMIN, CLIENTE
SalaEstado	DISPONIBLE, NO_DISPONIBLE
AsientoEstado	DISPONIBLE, MANTENIMIENTO

Enum	Valores
FuncionEstado	ACTIVA, CANCELADA
ReservaEstado	ACTIVA, CANCELADA, COMPLETADA, MODIFICADA

3.4 Mapeo Objeto-Relacional (JPA / Hibernate)

Entidades. Cada tabla se representa con una clase anotada con `@Entity` y `@Table`: Usuario, Sala, Asiento, Pelicula, Funcion, Reserva y DetalleReserva. Las claves primarias se mapean con `@Id` y `@GeneratedValue(strategy = GenerationType.IDENTITY)`, en correspondencia con las columnas GENERATED BY DEFAULT AS IDENTITY de Oracle.

Claves foráneas y relaciones. Las relaciones del modelo se expresan con anotaciones de asociación:

Relación	Mapeo
Asiento → Sala	<code>@ManyToOne</code> con <code>@JoinColumn(name = "sala_id")</code>
Funcion → Pelicula	<code>@ManyToOne</code> con <code>@JoinColumn(name = "pelicula_id")</code>
Funcion → Sala	<code>@ManyToOne</code> con <code>@JoinColumn(name = "sala_id")</code>
Reserva → Funcion	<code>@ManyToOne</code> con <code>@JoinColumn(name = "funcion_id")</code>
Reserva → Usuario	<code>@ManyToOne</code> con <code>@JoinColumn(name = "cliente_id")</code>
Reserva → DetalleReserva	<code>@OneToMany</code> por la clave <code>reserva_id</code> , con propagación de borrado
DetalleReserva → Asiento	<code>@ManyToOne</code> con <code>@JoinColumn(name = "asiento_id")</code>

El lado `@ManyToOne` es el propietario de la relación, y las asociaciones se declaran con carga diferida (`FetchType.LAZY`) para no cargar grafos de objetos innecesarios.

Enumeraciones. Los enums se mapean con `@Enumerated(EnumType.STRING)`, almacenando el nombre del valor como cadena (VARCHAR2). Así los datos son legibles en la base y el reordenamiento o la inserción de valores en el enum no corrompe los registros existentes, algo que sí ocurriría con `EnumType.ORDINAL`.

Tipos especiales.

- sinopsis se mapea como CLOB (`@Lob`) para textos extensos.
- `en_cartelera (NUMBER(1))` se proyecta a un Boolean en la entidad Pelicula (`enCartelera`).
- Los instantes (`fecha_hora_inicio`, `fecha_reserva`) se mapean a `LocalDateTime` sobre columnas `TIMESTAMP`.
- Los importes monetarios se mapean a `BigDecimal` para evitar errores de precisión de punto flotante.

Optimización de carga con `@EntityGraph`. Con asociaciones perezosas, recorrer una colección y acceder a su relación en cada elemento provoca el **problema de consultas N+1**: una consulta inicial más una consulta adicional por cada fila. Para resolverlo, los repositorios declaran `@EntityGraph(attributePaths = {...})` en los métodos de consulta que lo requieren (por ejemplo, funciones junto con su película y sala, o reservas junto con su función, cliente y butacas). Hibernate resuelve entonces el grafo en una sola consulta con `JOIN FETCH`.

Esto tiene una ventaja de diseño frente a declarar `FetchType.EAGER` en la entidad: la decisión de cuánto cargar se toma **por caso de uso** y no queda fija en el modelo, de modo que no se produce un acoplamiento temprano entre la entidad y una necesidad de lectura particular.

4. Patrones de Diseño de Software Aplicados

4.1 Arquitectura en Capas (Layered Architecture)

- **Qué es:** Organización del servidor en capas con responsabilidades únicas y dependencias en un solo sentido.
- **Dónde participa:** Cada módulo sigue la cadena Controller → Service → Repository (por ejemplo, ReservaController → ReservaService → ReservaRepository; del mismo modo para funciones, salas, películas y usuarios).
- **Justificación:** El controlador se limita a la interfaz HTTP, el servicio concentra las reglas de negocio y el repositorio solo accede a los datos. Un cambio en el formato de la API o en la base de datos no obliga a reescribir la lógica del dominio, y cada capa puede probarse de forma aislada.

4.2 Data Transfer Object (DTO)

- **Qué es:** Objetos dedicados a transportar datos entre cliente y servidor, distintos de las entidades persistentes.
- **Dónde participa:** ReservaRequest, ReservaResponse, FuncionRequest, SalaResponse, entre otros contratos de entrada y salida de la API.
- **Justificación:**
 - Desacopla el contrato público del esquema de la base de datos.
 - Evita exponer datos sensibles (como la contraseña de Usuario) y relaciones internas.
 - Permite validar la entrada en un único punto, con reglas distintas para cada operación.
 - Previene referencias circulares al serializar relaciones bidireccionales.

4.3 Repository Pattern

- **Qué es:** Una abstracción tipo colección sobre el acceso a datos, que oculta el mecanismo de persistencia.
- **Dónde participa:** Interfaces que extienden JpaRepository de Spring Data JPA, una por entidad principal (p. ej., ReservaRepository, FuncionRepository, SalaRepository, PeliculaRepository).
- **Justificación:** Las operaciones CRUD y las consultas derivadas por nombre de método se obtienen sin escribir SQL en la lógica de negocio. Los servicios dependen de una interfaz, no de Oracle, lo que facilita las pruebas y el cambio de motor.

4.4 Builder Pattern

- **Qué es:** Construcción paso a paso de objetos complejos con una interfaz fluida.
- **Dónde participa:** Lombok @Builder en entidades de dominio como Funcion, Pelicula y Sala.
- **Justificación:** Objetos con muchos atributos, varios opcionales, se crean con nombres explícitos para cada campo, evitando constructores largos con parámetros fáciles de intercambiar por error:

```

Funcion funcion = Funcion.builder()
    .pelicula(pelicula)
    .sala(sala)
    .fechaHoraInicio(inicio)
    .duracionMinutos(pelicula.getDuracionMinutos())
    .precioPorAsiento(precio)
    .estado(FuncionEstado.ACTIVA)
    .build();

```

4.5 Inyección de Dependencias e Inversión de Control (IoC)

- **Qué es:** El contenedor de Spring crea los objetos y entrega a cada clase sus colaboradores; las clases no los instancian por sí mismas.
- **Dónde participa:** Controladores y servicios con inyección **por constructor**, generada por Lombok con `@RequiredArgsConstructor` sobre atributos `final`.
- **Justificación:** Las dependencias son explícitas e inmutables, la clase nunca existe en un estado a medio configurar, y en las pruebas se sustituyen fácilmente por dobles (*mocks*). Las clases dependen de abstracciones (por ejemplo, la interfaz de un repositorio) y no de implementaciones concretas.

4.6 Single Page Application (SPA)

- **Qué es:** Una aplicación web que carga una sola vez y actualiza la interfaz dinámicamente, sin recargar la página completa.
- **Dónde participa:** Todo el cliente React: enrutamiento en el navegador, componentes del mapa de butacas, formularios y panel de administración.
- **Justificación:** La navegación entre cartelera, reserva y panel es fluida; el estado (sesión, butacas seleccionadas) se conserva en memoria; y los datos se obtienen con llamadas HTTP asíncronas (Axios) a la API REST. Esto es especialmente importante en el mapa de butacas, donde la selección debe responder al instante.

4.7 Resumen

Patrón	Capa / ámbito	Beneficio principal
Arquitectura en capas	Servidor	Separación de responsabilidades
DTO	Controladores ↔ cliente	Contrato de API estable y seguro
Repository	Persistencia	Acceso a datos sin SQL en el negocio
Builder	Dominio	Creación legible de objetos complejos
DI / IoC	Servidor	Bajo acoplamiento y testeabilidad
SPA	Cliente	Experiencia fluida sin recargas